phone_app/priv/repo/migrations/20231112032011_add_oban_jobs_table.exs

```
defmodule PhoneApp.Repo.Migrations.AddObanJobsTable do
  use Ecto.Migration

  def up do
    Oban.Migration.up(version: 11)
  end

  def down do
    Oban.Migration.down(version: 1)
  end
end
```

Then run mix ecto.migrate to run the migration.

This migration pattern puts all of the complex migrations behind an easy to use function. Plus, it makes it easy to handle future migrations as you keep Oban up-to-date over time.

Next, we need to add some configuration to set up our Oban queues. Add the following config block to config.exs:

phone_app/config/config.exs

```
config :phone_app, Oban,
  repo: PhoneApp.Repo,
  plugins: [
    # 1 hour
    {Oban.Plugins.Pruner, max_age: 60 * 60}
  ],
  queues: [default: 10]
```

The important options of this configuration are queues and plugins. Queues are buckets that jobs fall into. A given queue has a certain amount of concurrency available to it—in our case, ten workers can execute simultaneously. (This number is per server, so three servers of ten workers would provide thirty concurrent workers across a cluster.) You can define as many queues as you want and however much concurrency you want.

Plugins extend the capabilities of Oban. The Pruner plugin removes completed jobs from the database—in our case, after one hour.

Let's also update the test.exs config so that you have control over jobs in the test environment:

phone_app/config/test.exs

```
config :phone_app, Oban, testing: :manual
```

Finally, we need to add Oban to our application process list. Update the start function in lib/phone_app/application.ex to include the Oban Supervisor: